Desafio Prático: Robô de Auditoria Multitarefa

Objetivo: Criar um script unificado utilizando Selenium que opere em background (modo headless), gerencie múltiplas abas simultaneamente, combine esperas implícitas e explícitas para lidar com atrasos de rede e extraia informações visíveis e ocultas de elementos HTML.

Cenário: Você está desenvolvendo a base de um script de auditoria que, no futuro, fará parte de uma suíte de testes rodando com `pytest` em um servidor local com Armbian. Para otimizar o consumo de recursos dessa máquina e não exigir interface gráfica, a execução deve ser totalmente invisível. Lembre-se de configurar o WebDriver para utilizar o Chromium, garantindo compatibilidade com o ambiente.

Tarefas (Passo a Passo)

Parte 1: Configuração Inicial, Headless e Segurança de Rede (Espera Implícita)

1. Configure o WebDriver para rodar o Chromium em modo **Headless** (utilizando a flag `--headless=new`).
2. Defina o tamanho da janela do navegador para 1920x1080 para evitar que elementos fiquem ocultos por design responsivo.
3. Configure uma **Espera Implícita** global de 10 segundos logo após instanciar o driver. Isso servirá como uma "rede de segurança" contra oscilações comuns de rede durante toda a execução do script, garantindo que o Selenium faça o **polling** automático antes de falhar ao buscar elementos mais simples.

Parte 2: Orquestração de Abas

1. Ao iniciar o driver, acesse a URL: `https://the-internet.herokuapp.com/dropdown` (Esta será a aba 0).
2. Via injeção de script (`window.open`), abra uma nova aba e acesse `https://the-internet.herokuapp.com/dynamic_loading/2`. Aplique a validação necessária para garantir que a segunda aba foi aberta e reconhecida pelo Selenium antes de prosseguir.
   - Para validar que a nova aba foi aberta, verifique se o número de `window_handles` (identificadores de janela) aumentou antes de tentar mudar para a nova aba. Isso previne o comum erro `IndexError`.
3. Repita o processo para abrir uma terceira aba apontando para `https://pt.wikipedia.org`. Valide a abertura desta terceira janela de forma segura.

Parte 3: Interação com Dropdowns (Voltando à Aba 0)

1. Mude o foco do Selenium de volta para a primeira aba (Aba 0).
2. Localize o elemento `<select>` da página.
3. Utilizando a classe nativa do Selenium para este fim, selecione a opção **"Option 1"** utilizando o seu texto visível.
4. Utilizando o objeto da classe `Select`, acesse a propriedade que retorna a primeira opção selecionada (`first_selected_option`) e, a partir deste elemento, extraia seu atributo `value` para confirmar a seleção.

Parte 4: Sincronização Dinâmica (Espera Explícita na Aba 1)

1. Mude o foco do Selenium para a segunda aba (Aba 1).
2. Localize e clique no botão **"Start"**.
3. A página simulará um carregamento lento focado em um único elemento. Como a sua espera implícita global é de apenas 10 segundos, utilize uma **Espera Explícita** mais rigorosa, definindo um limite máximo de 15 segundos especificamente para que o elemento com o id `finish` se torne visível na tela. **Nota: Isso demonstra como combinar uma regra global (implícita) com exceções pontuais (explícita).**
4. Quando o elemento aparecer, extraia o texto dele e imprima no terminal.

Parte 5: Extração de Dados e Evidências (Aba 2)

1. Mude o foco do Selenium para a terceira aba (Aba 2 - Wikipédia). Graças à sua espera implícita configurada na Parte 1, o Selenium aguardará automaticamente caso os elementos da Wikipédia demorem frações de segundo para renderizar.
2. Localize a barra de pesquisa e digite a palavra "Automação".
3. Localize qualquer link do menu lateral de navegação (ex: "Artigo destacado").
4. Extraia e imprima no terminal:
   - O texto visível desse link.
   - O destino real para o qual o link aponta (atributo `href`).
5. Tire um print screen da tela inteira provando que a digitação na barra de pesquisa ocorreu com sucesso e salve com o nome `evidencia_wiki.png`.

Parte 6: Encerramento

1. Garanta que o método `quit()` seja chamado no final do script para encerrar o processo do navegador em background e não deixar processos zumbis na memória do servidor.

Dicas para o desenvolvimento:

- Lembre-se que o Selenium tenta executar os comandos instantaneamente. Se ocorrer o erro `IndexError: list index out of range` ao trocar de abas, revise como você está instruindo o robô a esperar pela criação da nova janela.
- Se um teste falhar silenciosamente por estar no modo headless, temporariamente tire a opção headless ou utilize o método `save_screenshot()` em pontos estratégicos do código para agir como os seus "olhos" e debugar o problema.